

BOOSTING TECHNIQUES

Complete Assignment Answers

Question 1: What is Boosting in Machine Learning? Explain how it improves weak learners.

Answer:

Boosting is an ensemble Machine Learning technique that combines multiple **weak learners** to create a strong predictive model.

A weak learner is a model that performs only slightly better than random guessing.

Boosting trains weak learners **sequentially**, where each new learner focuses on improving the mistakes or errors made by previous learners.

How Boosting Works:

1. Start with a simple/weak learner.
2. Evaluate its predictions.
3. Give more attention to incorrectly predicted observations or residual errors.
4. Train another weak learner to improve those errors.
5. Repeat the process for several iterations.
6. Combine all weak learners into one strong model.

Example:

Dataset

↓

Weak Learner 1



Find Errors



Weak Learner 2



Focus on Errors



Weak Learner 3



Combine Learners



Strong Model

Popular Boosting Algorithms:

- AdaBoost
- Gradient Boosting
- XGBoost
- CatBoost
- LightGBM

Main Advantage:

Boosting can reduce **bias** and improve predictive performance by gradually correcting the errors of earlier models.

Easy Definition:

Boosting combines many weak learners sequentially to create a strong learner.

Question 2: What is the Difference Between AdaBoost and Gradient Boosting in Terms of How Models Are Trained?

Answer:

AdaBoost and Gradient Boosting are both boosting techniques, but they train subsequent models differently.

AdaBoost

AdaBoost gives more weight to training observations that were **misclassified by previous learners**.

The next weak learner focuses more on difficult observations.

Process:

Model 1

↓

Identify Misclassified Samples

↓

Increase Their Importance

↓

Model 2

↓

Repeat

Gradient Boosting

Gradient Boosting trains each new model to predict the **residual errors or negative gradient of the loss function** produced by the current ensemble.

Process:

Initial Prediction

↓

Calculate Residuals

↓

Train Model on Residuals

↓

Update Prediction

↓

Calculate New Residuals

↓

Repeat

Comparison:

AdaBoost	Gradient Boosting
Focuses on misclassified/hard observations through weights	Fits residuals/negative gradients
Updates observation weights	Updates the ensemble using loss gradients
Often uses shallow Decision Trees	Commonly uses regression trees
Sensitive to noisy observations	Can also overfit without proper regularization
Sequential training	Sequential training

Easy Definition:

AdaBoost focuses on difficult observations; Gradient Boosting focuses on reducing the loss by fitting residual errors.

Question 3: How Does Regularization Help in XGBoost?

Answer:

Regularization helps XGBoost control model complexity and reduce overfitting.

XGBoost uses regularization terms that penalize overly complex trees.

Important parameters include:

1. `reg_alpha`

L1 regularization.

It can encourage some leaf weights toward zero and can help control model complexity.

2. `reg_lambda`

L2 regularization.

It penalizes large leaf weights and helps stabilize the model.

3. `max_depth`

Controls the maximum depth of individual trees.

Smaller depth generally produces simpler trees.

4. `min_child_weight`

Controls the minimum amount of information/weight needed in a child node before a split is allowed.

5. `gamma`

Controls the minimum loss reduction required to make a split.

Example:

```
from xgboost import XGBClassifier
```

```
model = XGBClassifier(  
  
    max_depth=4,  
  
    learning_rate=0.05,  
  
    reg_alpha=0.1,  
  
    reg_lambda=1.0  
)
```

Benefits of Regularization:

- Reduces overfitting.
- Controls model complexity.
- Improves generalization.
- Can make the model more robust.

Easy Definition:

Regularization prevents XGBoost from becoming unnecessarily complex and helps reduce overfitting.

Question 4: Why is CatBoost Considered Efficient for Handling Categorical Data?

Answer:

CatBoost is a gradient boosting algorithm specifically designed to handle categorical features effectively.

One of its important advantages is that it can process categorical variables directly without requiring the user to manually one-hot encode every categorical feature.

CatBoost uses specialized **ordered target statistics/encoding techniques** designed to reduce target leakage compared with naive target encoding.

Example:

Suppose the dataset contains:

City

Delhi

Mumbai

Jaipur

Delhi

CatBoost can work with the categorical column by specifying it as a categorical feature.

Python Example:

```
from catboost import CatBoostClassifier
```

```
model = CatBoostClassifier(
```

```
        verbose=0

    )

model.fit(

    X_train,

    y_train,

    cat_features=categorical_columns

)
```

Advantages:

- Handles categorical features directly.
- Reduces the need for extensive manual encoding.
- Works well with mixed data types.
- Provides strong predictive performance.
- Includes techniques designed to reduce target leakage during categorical processing.

Easy Definition:

CatBoost is efficient with categorical data because it has built-in techniques for processing categorical features.

Question 5: What are Some Real-World Applications Where Boosting Techniques Are Preferred Over Bagging Methods?

Answer:

Boosting techniques are often preferred when achieving high predictive accuracy is important and the additional sequential training complexity is acceptable.

1. Credit Risk Prediction

Banks can predict whether a customer is likely to default on a loan.

Algorithms:

- XGBoost
 - Gradient Boosting
 - CatBoost
-

2. Fraud Detection

Boosting can identify suspicious transactions based on transaction patterns.

3. Customer Churn Prediction

Companies can predict whether a customer is likely to leave.

4. Marketing

Boosting can predict:

- Customer response
 - Purchase probability
 - Campaign conversion
-

5. Insurance

Used for:

- Risk prediction
 - Claim prediction
 - Fraud detection
-

6. Healthcare

Used for:

- Disease-risk prediction
- Patient classification
- Clinical decision-support applications

Boosting vs Bagging:

Boosting	Bagging
Sequential learning	Models can be trained independently
Focuses on errors	Reduces variance through aggregation
Often reduces bias strongly	Mainly reduces variance

Can achieve very high accuracy

Usually more stable and easier to parallelize

Examples: XGBoost, AdaBoost

Example: Random Forest

Easy Definition:

Boosting is often preferred when maximizing predictive performance is more important than simplicity and training speed.

PART B – PRACTICAL QUESTIONS

Question 6: Train an AdaBoost Classifier on the Breast Cancer Dataset and Print Model Accuracy.

Answer:

The assignment specifies `load_breast_cancer()` for classification tasks.

Python Code:

```
from sklearn.datasets import load_breast_cancer

from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeClassifier

from sklearn.ensemble import AdaBoostClassifier

from sklearn.metrics import accuracy_score
```

```
# Load Breast Cancer Dataset

data = load_breast_cancer()

X = data.data

y = data.target

# Split dataset

X_train, X_test, y_train, y_test = train_test_split(

    X,

    y,

    test_size=0.2,

    random_state=42,

    stratify=y

)

# Base weak learner

base_model = DecisionTreeClassifier(

    max_depth=1,
```

```
        random_state=42

    )

# AdaBoost Classifier

model = AdaBoostClassifier(

    estimator=base_model,

    n_estimators=100,

    learning_rate=0.5,

    random_state=42

)

# Train model

model.fit(X_train, y_train)

# Predictions

y_pred = model.predict(X_test)

# Accuracy

accuracy = accuracy_score(
```

```
y_test,  
  
y_pred  
  
)  
  
print("AdaBoost Accuracy:", accuracy)
```

Output:

AdaBoost Accuracy: <value produced when the code is executed>

Observation:

The accuracy value depends on the Scikit-learn version and model settings. Run the code in Google Colab to obtain the exact output.

Question 7: Train a Gradient Boosting Regressor on the California Housing Dataset and Evaluate Using R-Squared Score.

Answer:

The assignment specifies `fetch_california_housing()` for regression tasks.

Python Code:

```
from sklearn.datasets import fetch_california_housing

from sklearn.model_selection import train_test_split

from sklearn.ensemble import GradientBoostingRegressor

from sklearn.metrics import r2_score


# Load California Housing Dataset

data = fetch_california_housing()


X = data.data

y = data.target


# Split dataset

X_train, X_test, y_train, y_test = train_test_split(

    X,

    y,

    test_size=0.2,

    random_state=42

)
```

```
# Gradient Boosting Regressor

model = GradientBoostingRegressor(

    n_estimators=100,

    learning_rate=0.05,

    max_depth=3,

    random_state=42

)
```

```
# Train model
```

```
model.fit(X_train, y_train)
```

```
# Predictions
```

```
y_pred = model.predict(X_test)
```

```
# R-squared
```

```
r2 = r2_score(
```

```
    y_test,
```

```
    y_pred
```

```
)
```

```
print("R-squared Score:", r2)
```

Output:

```
R-squared Score: <value produced when the code is executed>
```

Interpretation:

R^2 measures the proportion of variation in the target variable explained by the model.

A higher R^2 generally indicates better fit on the evaluated data.

Question 8: Train an XGBoost Classifier on the Breast Cancer Dataset, Tune Learning Rate Using GridSearchCV, and Print Best Parameters and Accuracy.

Answer:

First install XGBoost in Google Colab if required.

Step 1: Install XGBoost

```
!pip install -q xgboost
```

Step 2: Python Code

```
from sklearn.datasets import load_breast_cancer

from sklearn.model_selection import (

    train_test_split,

    GridSearchCV

)

from sklearn.metrics import accuracy_score

from xgboost import XGBClassifier


# Load dataset

data = load_breast_cancer()


X = data.data

y = data.target


# Split dataset

X_train, X_test, y_train, y_test = train_test_split(

    X,
```

```
        y,

        test_size=0.2,

        random_state=42,

        stratify=y

    )

# XGBoost Classifier

model = XGBClassifier(

    n_estimators=100,

    max_depth=3,

    random_state=42,

    eval_metric="logloss"

)

# Parameter grid

param_grid = {

    "learning_rate": [

        0.01,

        0.05,
```

```
        0.1,

        0.2

    ]

}

# GridSearchCV

grid = GridSearchCV(

    estimator=model,

    param_grid=param_grid,

    cv=5,

    scoring="accuracy",

    n_jobs=-1

)

# Train

grid.fit(

    X_train,

    y_train

)
```

```
# Best parameters

print("Best Parameters:")

print(grid.best_params_)


# Best model

best_model = grid.best_estimator_


# Predictions

y_pred = best_model.predict(

    X_test

)


# Accuracy

accuracy = accuracy_score(

    y_test,

    y_pred

)
```

```
print("\nTest Accuracy:", accuracy)
```

Output:

```
Best Parameters:
```

```
{'learning_rate': <value>}
```

```
Test Accuracy: <value>
```

Observation:

GridSearchCV tests the specified learning-rate values using 5-fold cross-validation and selects the value producing the best cross-validation accuracy.

Question 9: Train a CatBoost Classifier and Plot the Confusion Matrix Using Seaborn.

Answer:

CatBoost can directly handle categorical features, but the Breast Cancer dataset contains numerical features. Therefore, CatBoost can be trained directly without specifying categorical columns.

Step 1: Install CatBoost

```
!pip install -q catboost
```

Step 2: Python Code

```
from sklearn.datasets import load_breast_cancer

from sklearn.model_selection import train_test_split

from sklearn.metrics import confusion_matrix

from catboost import CatBoostClassifier


import seaborn as sns

import matplotlib.pyplot as plt


# Load Breast Cancer Dataset

data = load_breast_cancer()

X = data.data

y = data.target


# Split dataset

X_train, X_test, y_train, y_test = train_test_split(
```

```
    X,  
  
    y,  
  
    test_size=0.2,  
  
    random_state=42,  
  
    stratify=y  
)
```

```
# CatBoost Classifier
```

```
model = CatBoostClassifier(  
  
    iterations=200,  
  
    learning_rate=0.05,  
  
    depth=5,  
  
    verbose=0,  
  
    random_seed=42  
)
```

```
# Train
```

```
model.fit(  
  
    X_train,
```

```
        y_train

    )

# Predictions

y_pred = model.predict(

    X_test

).astype(int).ravel()

# Confusion Matrix

cm = confusion_matrix(

    y_test,

    y_pred

)

print("Confusion Matrix:")

print(cm)

# Visualization

plt.figure(figsize=(7, 5))
```

```
sns.heatmap(  
  
    cm,  
  
    annot=True,  
  
    fmt="d",  
  
    cmap="Blues"  
  
)  
  
plt.xlabel("Predicted")  
  
plt.ylabel("Actual")  
  
plt.title("CatBoost Confusion Matrix")  
  
  
plt.show()
```

Output:

The program will display a numerical confusion matrix and a heatmap.

Example format:

Confusion Matrix:

```
[[TN FP]
```

[FN TP]]

The exact values depend on the executed train/test split and model settings.

Observation:

The confusion matrix shows:

- **TN:** True Negatives
 - **FP:** False Positives
 - **FN:** False Negatives
 - **TP:** True Positives
-

Question 10: FinTech Loan Default Prediction Using Boosting Techniques

Problem Statement

A FinTech company wants to predict **loan default** using customer demographics and transaction behavior.

The dataset:

- Is imbalanced.
- Contains missing values.
- Contains numerical features.
- Contains categorical features.

The objective is to build a reliable boosting-based classification pipeline.

Step 1: Understand the Business Problem

The target variable can be:

Loan_Default

For example:

0 = No Default

1 = Default

The goal is to predict the probability that a customer will default on a loan.

Step 2: Load and Inspect the Dataset

```
import pandas as pd
```

```
df = pd.read_csv("loan_data.csv")
```

```
print(df.head())
```

```
print("\nDataset Information:")
```

```
print(df.info())

print("\nMissing Values:")

print(df.isnull().sum())

print("\nTarget Distribution:")

print(df["Loan_Default"].value_counts())
```

We should check:

- Number of rows and columns
 - Data types
 - Missing values
 - Duplicate records
 - Class distribution
 - Numerical variables
 - Categorical variables
 - Potential data leakage
-

Step 3: Separate Features and Target

```
X = df.drop(

    "Loan_Default",

    axis=1

)
```

```
y = df["Loan_Default"]
```

Here:

- x = input features
 - y = target variable
-

Step 4: Identify Numerical and Categorical Features

```
numeric_features = X.select_dtypes(  
  
    include=["int64", "float64"]  
  
).columns  
  
categorical_features = X.select_dtypes(  
  
    include=["object", "category", "bool"]  
  
).columns  
  
print("Numerical Features:")  
  
print(list(numeric_features))
```

```
print("\nCategorical Features:")

print(list(categorical_features))
```

Step 5: Split the Dataset

Because the target is imbalanced, use stratification.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(

    X,

    y,

    test_size=0.2,

    random_state=42,

    stratify=y

)
```

Why stratify=y?

It helps maintain approximately the same proportion of default/non-default cases in both training and testing data.

Step 6: Handle Missing Values

Numerical Variables

Use median imputation.

```
from sklearn.impute import SimpleImputer
```

```
numeric_imputer = SimpleImputer(  
  
    strategy="median"  
  
)
```

Categorical Variables

Use the most frequent category.

```
categorical_imputer = SimpleImputer(  
  
    strategy="most_frequent"  
  
)
```

The imputers should be fitted on the training data only and then applied to the test data.

Step 7: Encode Categorical Features

For models that require numerical inputs, categorical variables can be encoded.

```
from sklearn.preprocessing import OneHotEncoder
```

```
categorical_encoder = OneHotEncoder(  
  
    handle_unknown="ignore"  
  
)
```

`handle_unknown="ignore"` prevents errors if a category appears in the test set that was not present in the training set.

Step 8: Build a Preprocessing Pipeline

```
from sklearn.compose import ColumnTransformer
```

```
from sklearn.pipeline import Pipeline
```

```
numeric_pipeline = Pipeline(  
  
    (  
  
        "imputer",
```

```

        SimpleImputer(strategy="median")

    )

])

categorical_pipeline = Pipeline([

    (

        "imputer",

        SimpleImputer(strategy="most_frequent")

    ),

    (

        "encoder",

        OneHotEncoder(

            handle_unknown="ignore"

        )

    )

])

```

```

preprocessor = ColumnTransformer([

    (

```

```
        "num",

        numeric_pipeline,

        numeric_features

    ),

    (

        "cat",

        categorical_pipeline,

        categorical_features

    )

])
```

Step 9: Choose the Boosting Algorithm

There are three possible choices mentioned in the assignment:

AdaBoost

Advantages:

- Simple
- Effective with weak learners
- Easy to understand

Disadvantage:

- Can be sensitive to noisy observations.
-

XGBoost

Advantages:

- Strong predictive performance.
 - Regularization support.
 - Handles non-linear relationships.
 - Many tuning options.
 - Supports class weighting through parameters such as `scale_pos_weight`.
-

CatBoost

Advantages:

- Excellent support for categorical features.
- Less manual categorical preprocessing may be needed.
- Strong performance on mixed-type tabular data.

My Choice:

For this FinTech problem, **CatBoost or XGBoost** would be strong candidates.

If the dataset contains many categorical features, **CatBoost** is particularly attractive.

If maximum tuning flexibility and strong tabular performance are priorities, **XGBoost** is also an excellent choice.

Step 10: Handle Class Imbalance

Suppose the data contains:

No Default = 90%

Default = 10%

A model could achieve 90% accuracy by predicting "No Default" for almost everyone, which would be useless.

Therefore, class imbalance must be considered.

For XGBoost, one common approach is:

```
scale_pos_weight = negative_count / positive_count
```

Example:

```
from xgboost import XGBClassifier
```

```
negative_count = (y_train == 0).sum()
```

```
positive_count = (y_train == 1).sum()
```

```
scale_pos_weight = (  
  
    negative_count / positive_count  
  
)
```

```
model = XGBClassifier(  
  
    n_estimators=300,  
  
    learning_rate=0.05,  
  
    max_depth=4,  
  
    scale_pos_weight=scale_pos_weight,  
  
    eval_metric="logloss",  
  
    random_state=42  
  
)
```

Step 11: Train the Model

For XGBoost with preprocessing:

```
from sklearn.pipeline import Pipeline
```

```
xgb_pipeline = Pipeline([
```

```
(
```

```
    "preprocessor",
```

```
        preprocessor

    ),

    (

        "model",

        model

    )

])
```

```
xgb_pipeline.fit(

    X_train,

    y_train

)
```

Step 12: Hyperparameter Tuning

Important XGBoost hyperparameters include:

- `learning_rate`
- `n_estimators`
- `max_depth`
- `min_child_weight`

- subsample
- colsample_bytree
- gamma
- reg_alpha
- reg_lambda

Example:

```
from sklearn.model_selection import GridSearchCV
```

```
param_grid = {  
  
    "model__learning_rate": [  
  
        0.03,  
  
        0.05,  
  
        0.1  
  
    ],  
  
    "model__max_depth": [  
  
        3,  
  
        4,  
  
        6  
  
    ],  
  
    "model__n_estimators": [  
  
        100,  
  
    ]  
}
```

```
        200,  
        300  
    ]  
}  
  
grid_search = GridSearchCV(  
    xgb_pipeline,  
    param_grid,  
    cv=5,  
    scoring="roc_auc",  
    n_jobs=-1  
)  
  
grid_search.fit(  
    X_train,  
    y_train  
)  
  
print("Best Parameters:")
```

```
print(grid_search.best_params_)
```

Why ROC-AUC for Tuning?

Because the dataset is imbalanced, ROC-AUC is often more informative than raw accuracy for model selection. For heavily imbalanced data, PR-AUC can also be particularly useful.

Step 13: Generate Predictions

```
y_pred = grid_search.predict(  
  
    X_test  
  
)
```

```
y_prob = grid_search.predict_proba(  
  
    X_test  
  
)[:, 1]
```

`y_prob` contains the predicted probability of loan default.

Step 14: Evaluate the Model

Use several metrics.

```
from sklearn.metrics import (

    accuracy_score,

    precision_score,

    recall_score,

    f1_score,

    roc_auc_score,

    average_precision_score,

    confusion_matrix,

    classification_report

)


print(

    "Accuracy:",

    accuracy_score(y_test, y_pred)

)


print(

    "Precision:",
```

```
precision_score(y_test, y_pred)

)
```

```
print(

    "Recall:",

    recall_score(y_test, y_pred)

)
```

```
print(

    "F1-Score:",

    f1_score(y_test, y_pred)

)
```

```
print(

    "ROC-AUC:",

    roc_auc_score(y_test, y_prob)

)
```

```
print(
```

```
        "PR-AUC:",  
  
        average_precision_score(y_test, y_prob)  
  
    )  
  
    print("\nConfusion Matrix:")  
  
    print(  
  
        confusion_matrix(  
  
            y_test,  
  
            y_pred  
  
        )  
  
    )
```

Step 15: Why These Metrics?

Accuracy

Measures overall correct predictions.

However, it can be misleading with imbalanced loan-default data.

Precision

Answers:

Of all customers predicted as defaulters, how many actually defaulted?

High precision reduces false alarms.

Recall

Answers:

Of all actual defaulters, how many did the model identify?

High recall is important when missing a potential defaulter is costly.

F1-Score

Balances Precision and Recall.

$$F1 = 2 \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

ROC-AUC

Measures the model's ability to rank positive cases above negative cases across thresholds.

PR-AUC

Especially useful when the positive class is relatively rare because it focuses directly on the precision-recall trade-off.

Step 16: Business Value of the Model

A successful loan-default prediction model can provide significant business value.

1. Reduce Financial Loss

The company can identify high-risk customers before approving loans or increasing credit exposure.

2. Improve Credit Decisions

The model can provide a risk score that supports lending decisions.

3. Better Risk-Based Pricing

Higher-risk customers may be assigned different loan terms or interest rates according to the company's policies and applicable regulations.

4. Improve Customer Segmentation

Customers can be grouped into:

Low Risk

Medium Risk

High Risk

5. Improve Fraud/Risk Monitoring

Unusual transaction behavior can be used as one of the signals for further investigation.

6. Better Resource Allocation

Risk teams can prioritize customers who require additional review.

7. Reduce Default Rate

Better identification of high-risk applications can potentially reduce future loan losses.

Complete FinTech Machine Learning Pipeline

Raw Customer Data

↓

Data Understanding

↓

Train-Test Split

↓

Missing Value Handling

↓

Categorical Encoding



Class Imbalance Handling



Choose Boosting Algorithm



Train Model



Hyperparameter Tuning



Cross-Validation



Predictions



Precision / Recall / F1

ROC-AUC / PR-AUC



Business & Risk Validation



Deployment



Continuous Monitoring

Important Practical Considerations

For a real FinTech deployment, the following should also be considered:

Data Leakage

Information that would only become available after the loan outcome should not be used as a feature.

Fairness

The model should be checked for unfair performance differences across relevant customer groups.

Explainability

Credit decisions may require interpretable reasons and appropriate documentation depending on the jurisdiction and business process.

Probability Calibration

If the model's output is used as a risk probability, calibration should be evaluated.

Monitoring

After deployment, monitor:

- Data drift
- Prediction drift
- Default rate
- Model performance
- Calibration
- Fairness
- Business impact

FINAL REVISION TABLE

Topic	Key Point
Boosting	Combines weak learners sequentially
Weak Learner	Model with relatively weak predictive performance
AdaBoost	Focuses more on incorrectly classified observations
Gradient Boosting	Fits residuals/negative gradients
XGBoost	Regularized and optimized gradient boosting
CatBoost	Strong support for categorical data
Learning Rate	Controls contribution of each boosting step
<code>n_estimators</code>	Number of boosting iterations/trees
<code>max_depth</code>	Controls tree complexity
<code>reg_alpha</code>	L1 regularization
<code>reg_lambda</code>	L2 regularization
GridSearchCV	Hyperparameter tuning
Accuracy	Overall correctness

Precision	Correct positive predictions among predicted positives
Recall	Correct positives among actual positives
F1	Balance of Precision and Recall
ROC-AUC	Ranking/discrimination metric
PR-AUC	Useful for imbalanced classification
Class Imbalance	Unequal distribution of target classes
Data Leakage	Unfair use of information unavailable at prediction time

IMPORTANT FORMULAS

F1-Score

$$F1 = 2 \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Precision

$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall

$$\text{Recall} = \frac{TP}{TP + FN}$$

TP+FN

R-Squared

$$R^2 = 1 - \frac{S_{\text{res}}}{S_{\text{tot}}}$$

CONCLUSION

Boosting is one of the most powerful ensemble learning approaches for tabular Machine Learning problems. It improves predictive performance by building models sequentially and focusing each new model on reducing the errors of the existing ensemble.

AdaBoost focuses on difficult observations, **Gradient Boosting** minimizes a loss function by fitting residuals/gradients, **XGBoost** adds regularization and engineering improvements, and **CatBoost** provides strong handling of categorical features.

For real-world applications such as FinTech loan-default prediction, boosting can help identify high-risk customers, reduce financial losses, improve credit decisions, and support risk management. However, model performance should be evaluated using appropriate metrics, especially when the dataset is imbalanced.
